Cuckoo search optimization

Code:

```
import random
import math

# Spring design objective function
def objective(x):
    w, d, L = x
    return (L + 2) * w**2 * d

# Four constraints from the research paper
def constraints(x):
    w, d, L = x

    g1 = 1 - (d**3 * L) / (7178 * w**4)
    g2 = 1 - (140.45 * w) / (d**2 * L)
    g3 = 2 * (w + d) / 3 - 1
    g4 = (
        d * (4 * d - w) / (w * (12566 * d - w))
        + 1 / (5108 * w**2) - 1
    )
```

```
    return [g1, g2, g3, g4]

# Penalize constraint violations
def fitness(x):
    penalty = sum(max(0, g)**2 for g in constraints(x))
    return objective(x) + 1e6 * penalty

# Lévy flight using Mantegna's method
def levy_flight(beta=1.5):
    sigma = (
        math.gamma(1 + beta) * math.sin(math.pi * beta / 2)
        / (
            math.gamma((1 + beta) / 2)
            * beta * 2**((beta - 1) / 2)
        )
    ) ** (1 / beta)

    u = random.gauss(0, sigma)
    v = random.gauss(0, 1)
    return u / (abs(v)**(1 / beta) + 1e-12)

# Design-variable bounds: w, d, L
lower = [0.05, 0.25, 2.0]
```

```
upper = [2.0, 1.3, 15.0]

def random_nest():
    return [
        random.uniform(lower[i], upper[i])
        for i in range(3)
    ]

def cuckoo_search(n=30, iterations=3000, pa=0.25):
    nests = [random_nest() for _ in range(n)]
    best = min(nests, key=fitness)

    # Preserve the best solution found
    for _ in range(iterations):
        for i in range(n):
            step = levy_flight()
            candidate = [
                nests[i][j] + 0.01 * (upper[j] - lower[j]) * step
                for j in range(3)
            ]

            candidate = [
                max(lower[j], min(upper[j], candidate[j]))
                for j in range(3)
```

```
            ]

            # Compare with a randomly selected nest
            k = random.randrange(n)
            if fitness(candidate) < fitness(nests[k]):
                nests[k] = candidate

        # Abandon some of the worst nests
        count = max(1, int(pa * n))
        worst = sorted(range(n), key=lambda i: fitness(nests[i]),
                       reverse=True)

        for i in worst[:count]:
            nests[i] = random_nest()

        current = min(nests, key=fitness)
        if fitness(current) < fitness(best):
            best = current[:]

        # Keep the best solution in the population
        worst_index = max(range(n), key=lambda i:
fitness(nests[i]))
        nests[worst_index] = best[:]
```

```
    return best

best = cuckoo_search()
w, d, L = best

print("SPRING DESIGN USING CUCKOO SEARCH")
print("------------------------------")
print(f"Wire diameter (w)       = {w:.7f}")
print(f"Mean coil diameter (d)  = {d:.7f}")
print(f"Length/coils variable L = {L:.7f}")
print(f"Objective value         = {objective(best):.9f}")
print("Constraint values       =", constraints(best))

if all(g <= 1e-7 for g in constraints(best)):
    print("Constraint status       = FEASIBLE")
else:
    print("Constraint status       = NOT FEASIBLE")
    print("Try increasing iterations or changing the random seed.")

print("Nischal AR")
```

OUTPUT:

```
    print("Constraint status       = FEASIBLE")
else:
    print("Constraint status       = NOT FEASIBLE")
    print("Try increasing iterations or changing the random seed.")


print("Nischal AR")
```

```
SPRING DESIGN USING CUCKOO SEARCH
---------------------------------
Wire diameter (w)       = 0.0500000
Mean coil diameter (d)  = 0.2500000
Length/coils variable L = 2.8712098
Objective value         = 0.003044506
Constraint values       = [-3.4247315434665637e-06, -38.13332933912009, -0.8, -0.9201794237514305]
Constraint status       = FEASIBLE
Nischal AR
```